

Unit - 5

Multi Threading GUI Programming, ~~and~~ Event Handling

Thread, Thread life Cycle (Thread Class, Runnable interface), Thread Synchronization, Inter Thread Communication.

Java GUI Basics, Introduction to AWT (Abstract Window Toolkit) and Swing, Layout managers, Event delegation model. Handling GUI events: Action listener, mouse listener. Building basic forms and interfaces with button, ~~for~~ labels, text fields.

1. Swing:

Swing is a part of Java foundation classes JFC. that is used to create graphical user Interface applications.

It is built on the top of AWT (Abstract Window Toolkit).

It is more powerful, flexible and Lightweight.

Swing provides a rich set of components like button, text field

table etc.

All the swing classes are defined in `javax.swing` package.

The components in Swing:

i) JFrame:

JFrame is a class in `javax.swing` package, that represents a top level window with a title bar, border, close, maximize and minimize buttons.

Ex:

```
import javax.swing.*;
```

```
class framedemo
```

```
{
```

```
    public static void main(String args)
```

```
{
```

```
        JFrame f = new JFrame("Welcome");
```

```
        f.setSize(300, 300);
```

output

```
        f.setVisible(true);
```

```
    }
```

```
}
```



Ex: 2)

```
import javax.swing.*;
```

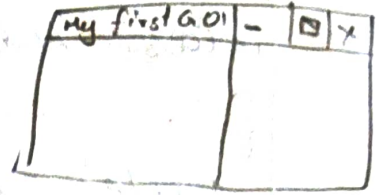
```
class frame
```

```
{
```

```
    public static void main(String args)
```

```
{
```

```
JFrame f = new JFrame ("My first GUI");
f.setSize(300, 300);
f.setVisible(true);
```



Components:

JLabel - It displays a text or image.

JTextField - It is used to create clickable buttons.
It allows the user to enter single line of text.

TextArea - It allows the user to enter multi line of text.

JButton

JCheckBox - The check box is used for multi choice selection.

JRadioButton - The radiobutton is used for single choice selection.

JList - It is used to display a list of items.

JMenu, JMenuItem - It is used to create menu and menu item.

J-Labels:

JLabel is a GUI component in swing used to display a short string of text, an image or both.

Constructor:

i) `JLabel()`

ii) `JLabel(String text)`

iii) `JLabel(Icon image)`

iv) `JLabel(String text, Icon image)`

Ex:

```
import javax.swing.*;
```

```
class LabelDemo
```

```
{  
    public static void main(String args[])  
    {
```

```
        JFrame a = new JFrame("my first GUI""GUI");
```

```
        U.A. frame a.setSize(700, 400);
```

```
        a.setVisible(true);
```

```
        JLabel l1 = new JLabel("Name");
```

```
        JLabel l2 = new JLabel("Roll no");
```

```
        f.add(l1);
```

```
        f.add(l2);
```

```
    }
```

```
}
```

JTextField:

JTextField is a swing component used to enter and edit a single line of text. It is commonly used for inputs like user name, email, numbers, search boxes etc.

It allows user inputs.

Constructors:

JTextField ()

JTextField (String Text)

JTextField (int columns)

JTextField (String text, int column)

JButtons:

JButton is a swing component that represents a push button. When the user click it an action event is triggered.

* It can display text, icon or both.

* It is often use for summing form, triggering action, opening dialogues. etc.

Constructors:

JButton ()

JButton (String Text)

JButton (Icon image)

JButton (String text, Icon Image)

Example:

A hand-drawn diagram of a Java Swing window. The window has a title bar with three buttons: a minus sign, a square, and an 'X'. Inside the window, there are two labels: 'name:' and 'Roll no:'. Each label is followed by a rectangular text input field. The 'Roll no:' field contains the text '10100'. At the bottom of the window, there are two buttons: 'clear' and 'ok'.

```
de: import javax.swing.*;  
class Swingdemo
```

```
{  
    public static void main(String args[])  
    {
```

```
        JFrame f = new JFrame("Registration  
                                form");
```

```
        f.setSize(300, 300);
```

```
        f.setVisible(true);
```

```
        JLabel l1 = new JLabel("Name")
```

```
        JTextField t1 = new
```

```
        JLabel l2 = new JLabel("Roll no");
```

```
        JTextField t2 = new
```

```
        JTextField t1 = new JTextField("");
```

```
        JTextField t2 = new JTextField("");
```

```
        JButton b1 = new JButton("Leave");
```

```
        JButton b2 = new JButton("Ok");
```

```
        f.add(l1);
```

```
        f.add(t1);
```

```
        f.add(l2);
```

```
        f.add(t2);
```

```
        f.add(b1);
```

f.add(b);

}

}

Layout Manager :-

In java awt and swing layout manager is an object that controls the size, position and arrangement of component.

Instead of manually setting the position layout manager automatically adjust the component placement.

It is defined in the package java.awt.

Types of Layout Manager in Java:

- i) Flow Layout
- ii) Border Layout
- iii) Grid Layout
- iv) Card Layout
- v) Grid Bag Layout.

Flow Layout:

It arranges the component in a row from left to right.

When the space is insufficient it move the component to the next line.

It uses the alignment left, right and center.

Syntax:

```
FlowLayout fl = new FlowLayout();  
f.setLayout(fl);
```

Ex:

```
import java.awt.*;  
import javax.swing.*;  
class layoutdemo  
{  
    public static void main(String args[])
```

```
{  
    JFrame f = new JFrame ( );
```

```
    f.setSize (300, 300);  
    f.setVisible(False True);
```

```
    JButton b1 = new JButton("Button1");
```

```
    JButton b2 = new JButton("Button2");
```

```
    JButton b3 = new JButton("Button3");
```

```
    JButton b4 = new JButton("Button4");
```

```
    FlowLayout fl = new FlowLayout();
```

```
    f.setLayout(fl);
```

```
    f.add(b1);
```

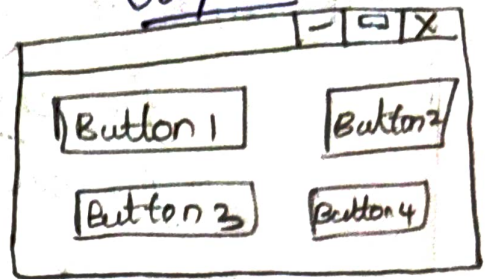
```
    f.add(b2);
```

f.add(b3);

f.add(b4);

3 3

Output:



Border Layout:-

It divides the container in five regions that is north, south, east, west and centre. It can add only one component per region. The centre expands to fill the unused space.

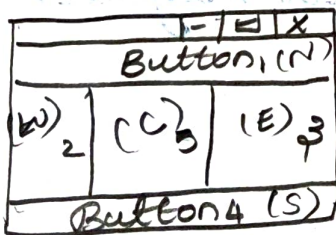
Syntax:

BorderLayout Bl = new BorderLayout();

f.setLayout(Bl);

f.add(b1, BorderLayout.SOUTH);

Ex:



code:

```
import java.awt.*;
```

```
import javax.swing.*;
```

```
class Borderdemo
```

```
{
```

```
    public static void main(String args[])
```

```
{
```

```
JFrame f = new JFrame ( );  
f.setSize(300,300);  
f.setVisible (True);
```

```
JButton b1 = new JButton ("Button 1");  
JButton b2 = new JButton ("Button 2");  
JButton b3 = new JButton ("Button 3");  
JButton b4 = new JButton ("Button 4");  
JButton b5 = new JButton ("Button 5");  
BorderLayout bl = new BorderLayout;  
f.setLayout (bl);
```

```
f.add (b1, BorderLayout.NORTH);  
f.add (b2, BorderLayout.WEST);  
f.add (b3, BorderLayout.EAST);  
f.add (b4, BorderLayout.SOUTH);  
f.add (b5, BorderLayout.CENTRE);  
}  
}
```

GridLayout:-

It divide the container into grid of row and columns.

All This else created by grid layout will be of equal size.

```
JFrame f = new JFrame ( );  
f.setSize(300,300);  
f.setVisible (True);
```

```
JButton b1 = new JButton ("Button 1");  
JButton b2 = new JButton ("Button 2");  
JButton b3 = new JButton ("Button 3");  
JButton b4 = new JButton ("Button 4");  
JButton b5 = new JButton ("Button 5");  
BorderLayout bl = new BorderLayout();  
f.setLayout (bl);
```

```
f.add (b1, BorderLayout.NORTH);  
f.add (b2, BorderLayout.WEST);  
f.add (b3, BorderLayout.EAST);  
f.add (b4, BorderLayout.SOUTH);  
f.add (b5, BorderLayout.CENTER);  
}  
}
```

GridLayout:-

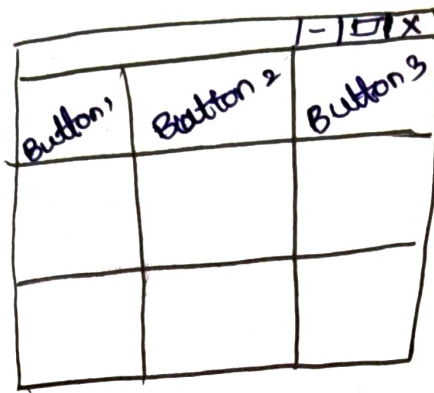
It divide the container into grid of row and columns,

All This else created by grid layout will be of equal

Syntax:

```
GridLayout { G = new GridLayout(3,3)  
f. SetLayout(G);
```

Eg:



Code:

```
import java.awt.*;  
import javax.swing.*;  
class GridDemo {
```

```
{  
    public static void main(String args[])  
    {
```

```
        JFrame f = new JFrame();
```

```
        f.setSize(300, 300);
```

```
        f.setVisible(true);
```

```
        JButton b1 = new JButton("Button 1");
```

```
        JButton b2 = new JButton("Button 2");
```

```
        JButton b3 = new JButton("Button 3");
```

```
        JButton b4 = new JButton("Button 4");
```

```
        JButton b5 = new JButton("Button 5");
```

```
        JButton b6 = new JButton("Button 6");
```

```
        JButton b7 = new JButton("Button 7");
```

```
        JButton b8 = new JButton("Button 8");
```

```
        JButton b9 = new JButton("Button 9");
```

```
GridLayout G = new GridLayout(3,3);  
f.setLayout(G);
```

```
f.add(b1);  
f.add(b2);  
f.add(b3);  
f.add(b4);  
f.add(b5);  
f.add(b6);  
f.add(b7);  
f.add(b8);  
f.add(b9);  
}  
}
```

Card Layout:

It stacks the components like deck of cards. Only one component is visible at a time, It is useful to wizards or tabbed UI.

Eg:

```
import java.awt.*;  
import javax.swing.*;  
class carddemo
```

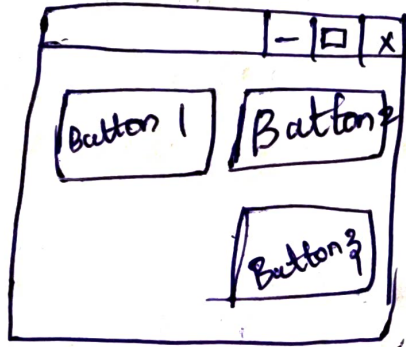
GridBag Layout:

It is most flexible and complex layout manager. It places a component in a grid but unlike grid layout here the cells can have different layout.

It is controlled using GridBag Constraint.

Eg:

Output:



Program:

```
import java.awt.*;  
import javax.swing.*;  
class gridbagdemo  
{  
    public static void main(String args[])  
    {
```

```
        JFrame f = new JFrame();
```

```
        f.setSize(300, 300);
```

```
        f.setVisible(true);  
    }
```

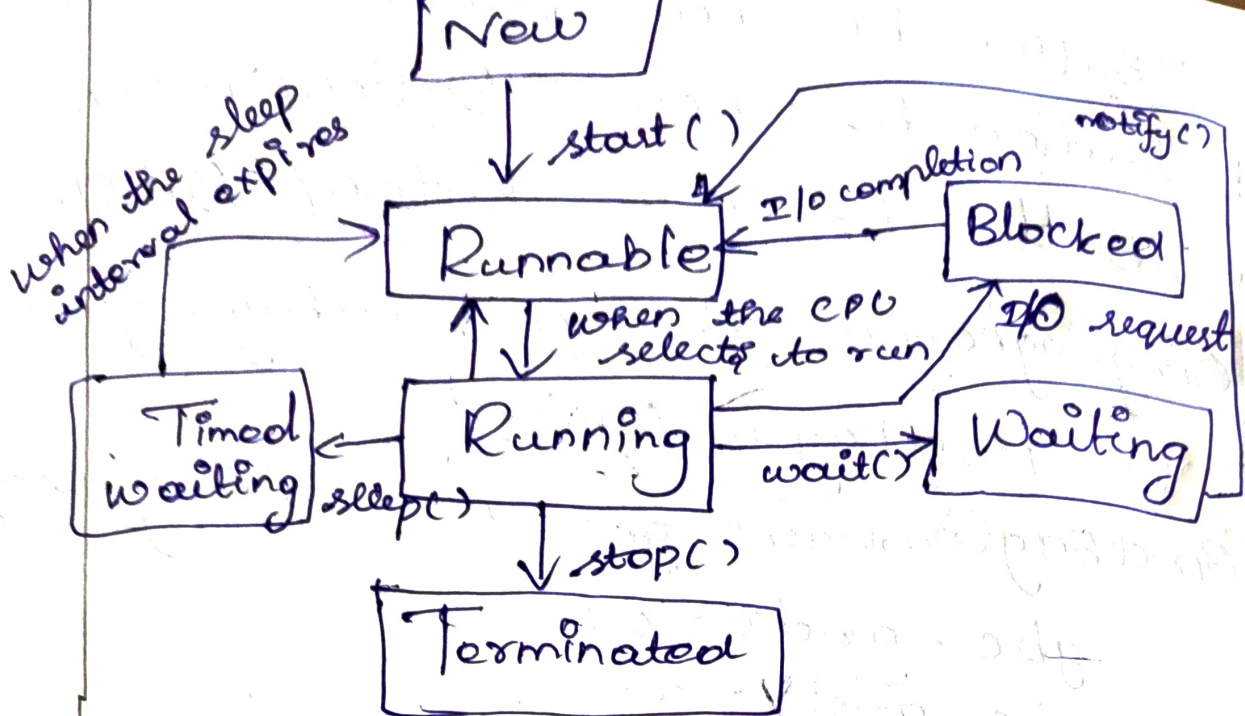
```

JButton b1 = new JButton("Button 1");
JButton b2 = new JButton("Button 2");
JButton b3 = new JButton("Button 3");
GridBagLayout gbl = new GridBagLayout();
f.setLayout(gbl);
GridBagConstraints gbc = new GridBagConstraints();
gbc.gridx = 0;
gbc.gridy = 0;
f.add(b1, gbc);
gbc.gridx = 0;
gbc.gridy = 1;
f.add(b2, gbc);
gbc.gridx = 1;
gbc.gridy = 1;
f.add(b3, gbc);
}
}

```

Thread Life Cycle:

In Java thread represents a separate part of execution the lifecycle of a thread is controlled by the thread class and runnable interface.



New state:

When a thread is created but not yet started then it is in the new state.

Syntax:

Thread t = new Thread();

Runnable state:

After calling start() of function the thread is get ready to run but waiting for CPU scheduling. This state is called as runnable state.

t.start() // Moves the thread to the runnable state.

Running state:

The Thread is in this stage when the CPU ^{Pick} fixed it up from the runnable pool and execute it

by using `run()` method. Only one thread can run at a time.

Waiting state:

The thread is in the waiting state when it calls the method `wait()` or `join()`.

In this state the thread is waiting indefinitely for another thread to perform a specific action:

Timed waiting:

It is similar to waiting state a thread enters in timed waiting state when it call a method with a time limit.

eg: `sleep(1000)`

Terminated state:

A thread enters the terminated state when it has completed its execution. At this point the thread is no longer alive and cannot be restarted.

Thread Creation:

There are 2 main way to create a thread in java.

- i) By extending thread class.
- ii) By implementing `Runnable` interface.

By Extending thread class:

A thread can be created by extending thread class and by overriding the run() method.

We can create an object by calling the start() function.

Thread Creation:

```
class mythread extends Thread
{
    public void run()
    {
        System.out.println("Thread running: " +
            Thread.currentThread().getName());
    }
}

public class ThreadExample
{
    public static void main(String args[])
    {
        mythread t1 = new mythread();
        mythread t2 = new mythread();
        t1.start();
        t2.start();
    }
}
```

O/P

Thread running: Thread-0

Thread running: Thread-1

By implementing runnable interface:

In this method create a class that implements runnable and override the run() method.

Create an object for a thread and pass it to runnable interface to create a complete thread.

Eg:

```
class myRunnable implements Runnable
{
    public void run()
    {
        System.out.println("Thread running: " +
            Thread.currentThread().getName());
    }
}

public class ThreadExample
{
    public static void main(String args[])
    {

```

```
myRunnable m = new myRunnable();
```

```
Thread t1 = new Thread(m);
```

```
Thread t2 = new Thread(m);
```

```
t1.start();
```

```
t2.start();
```

```
}
```

```
}
```

O/P

Thread Running : Thread-0

Thread Running : Thread-1

Thread Synchronization and Inter

Thread Communication:

Thread Synchronization:

Synchronization is a mechanism used to control access to a shared resources when multiple threads are running at the same time.

It ensures that only one thread access the critical section at the time.

Java provides the ^{ed}synchronized keyword to achieve synchronization. It can be used with methods and blocks.

Synchronized meth

A synchronized method is the method that allows only one thread at a time to execute it for the same object.

It is used when the entire method needs protection from multiple threads.

Syntax:

Synchronized void display() { // statement }

Eg: for syn

```
class demo {  
    synchronized void display()  
    {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println(i);  
        }  
    }  
    public static void main (String[] args) {  
        Demo d = new Demo();  
        Thread t1 = new Thread(() -> d.display());  
    }  
}
```

Thread t2 = new Thread(1) → d.display();

```
t1.start();  
t2.start();  
}  
}
```

Synchronized Block:

A synchronize block is use to synchronize only a specific section of a method. So it is useful when only a small part of method need synchronization.

Syntax:

```
synchronized (this) { // statement }
```

Eg:

```
class demo  
{  
    void display()  
    {  
        synchronized (this) {  
            for (int i = 1; i <= 5; i++)  
            {  
                System.out.println(i);  
            }  
        }  
    }  
}
```

```

public static void main(String args[])
{
    demo d = new demo();
    Thread t1 = new Thread(() -> d.display());
    Thread t2 = new Thread(() -> d.display());

    t1.start();
    t2.start();
}
}

```

Inter thread communication:-

Inter thread communication is the mechanism that allows one thread to communicate with another thread.

It is mainly used when one thread need to wait for another thread to complete some work.

Java provides 3 ^{method} ~~thread~~ for Inter thread communication.

i) wait(): It makes the thread wait.

ii) notify(): It wakes up one waiting threads

iii) notifyall(): It wake up all the waiting threads.

Eg:

```
class demo {  
    synchronized void display() {  
        System.out.println("Waiting...");  
        try {  
            wait();  
        } catch (Exception e) {  
            - }  
        System.out.println("Running...");  
    }  
    synchronized void notifyThread() {  
        System.out.println("Notify");  
        notify();  
    }  
    public static void main(String[] args) {  
        Demo d = new Demo();  
        Thread t1 = new Thread(() -> d.display());  
        Thread t2 = new Thread(() -> d.notifyThread());  
        t1.start();  
        t2.start();  
    }  
}
```

Output:

Waiting.... Notify Running.....

Action Listeners:

Action Listeners is an interface in java.awt.event package. It is used to handle action event such as clicking a button, choosing a menu item and pressing enter in text field.

Whenever the user performs an action event, action listener, action performed() method is executed.

Syntax of Action Listener:

```
import java.awt.event. ActionListener;  
import java.awt.event. ActionEvent;  
{  
    public interface ActionListener extends  
        EventListener  
    {  
        void actionPerformed(ActionEvent e);  
    }  
}
```

Eg: i)

```
import javax.swing.*;  
import java.awt.event.*;  
class ActionListenerDemo implements ActionListener
```

```
{  
    public static void main(String, o  
        ActionListenerDemo  
        JFrame f = new JFrame();
```

```
    f.setSize(300, 300);
```

```
    f.setVisible(true);
```

```
    JButton b = new JButton("Click me");
```

```
    b.addActionListener(this);  
}
```

```
public void actionPerformed(ActionEvent e)  
{
```

```
    JOptionPane.showMessageDialog("Button  
        clicked");  
}
```

```
public static void main(String args[])  
{
```

```
    ActionListenerDemo d = new ActionListenerDemo();
```

```
}
```

actionPerformed(e)

showMessageDialog

ii) Eg:

```
import javax.swing.*;*;  
import java.awt.event.*;  
class action implements ActionListener  
{  
    action  
{  
        JFrame f = new JFrame();  
        f.setSize(300, 300);  
        f.setVisible(true);  
        JButton b = new JButton("Greet");  
        f.add(b);  
        b.addaddActionListener(this);  
    }  
    public void actionPerformed(ActionEvent e)  
    {  
        JOptionPane.showMessageDialog  
            ("Welcome  
            to my home");  
    }  
    public static void main(String args[])  
    {  
        action a = new action();  
    }  
}
```

MouseListener:

MouseListener is an interface in java use to handle mouse events such as clicking, pressing, releasing, entering and exiting a component.

MouseListener Components:

Method	Ret	Purpose
1. mouseClicked()		It called when the mouse is clicked
2. mousePressed()		It is called when a mouse button is pressed
3. mouseReleased()		It is called when a mouse button is released
4. mouseEntered()		It is called when the mouse enters a component
5. mouseExited()		It is called when the mouse leaves the component.

Eg:

```
import java.swing.*;
import java.awt.*;
class mousedemo implements MouseListener
{
    mousedemo
    {
        JFrame f = new JFrame();
        f.setSize(300, 300);
        f.setVisible(true);
        JLabel L1 = new JLabel();
        f.add(L1);
        f.addMouseListener(this);
    }
    public void mouseClicked(MouseEvent e)
    {
        L1.setText("Mouse clicked");
    }
    public void mousePressed(MouseEvent e)
    {
        L1.setText("Mouse pressed");
    }
}
```

```
public void mouseReleased (Mouse Event e)
{
    l1.setText ("Mouse Released");
}
public void mouseEntered (Mouse Event e)
{
    l1.setText ("Mouse Entered");
}
public void mouseExited (Mouse Event e)
{
    l1.setText ("Mouse Exited");
}
public static void main (String args[])
{
    mousedemo # d = new mousedemo();
}
}
```